

Deep Learning for Solving Dynamic Models

Session 6: Deep surrogates and Gaussian processes

Simon Scheidegger

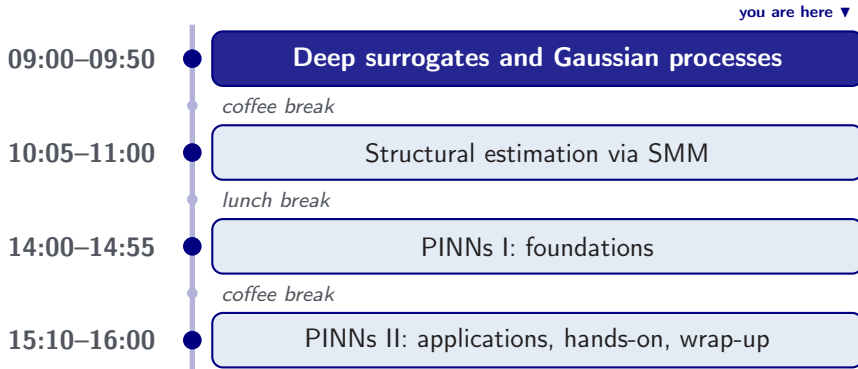
HEC, University of Lausanne

Based on Chen, Didisheim & Scheidegger (2026), *Journal of Financial Economics* 177, 104222

University of Padova

September 24, 2026

Day 2 at a Glance



Roadmap of This Lecture (90 min)

I. What Is a Surrogate? (12 min)

- Look-up tables, pseudo-states, and what the speed-up buys

II. Estimate: structural estimation (20 min)

- SMM on a surrogate; Brock–Mirman; the identification ridge

III. Gaussian Processes (24 min)

- When data is expensive, a deep net is the wrong tool
- Kernels, credible bands, and active learning

IV. Quantify: uncertainty (10 min)

- Univariate effects, Sobol' indices, Shapley values for the SCC

V. Design: the optimal carbon tax (16 min)

- Policy coefficients as pseudo-states; 500 solves, not a million

VI. Synthesis and Hands-On (8 min)

- When to use what; the notebooks

Learning Objectives

By the end of this lecture you should be able to:

- ▶ Explain a surrogate as a **differentiable look-up table** and say precisely which classical bottleneck it removes
- ▶ Promote structural or policy parameters to **pseudo-states**, so that one solve answers a whole family of questions
- ▶ Estimate a structural parameter by simulated method of moments on a trained surrogate, and read an identification ridge off the criterion surface
- ▶ Turn the same surrogate into a global sensitivity analysis: univariate effects, Sobol' indices, Shapley values
- ▶ Fit a Gaussian process, read its credible band, and **decide between a GP and a deep network** from the cost of one model evaluation
- ▶ Place the next expensive evaluation where it buys the most, using Bayesian active learning

Part I

What Is a Surrogate?

The Bottleneck Is Always the Same

Expensive to evaluate

One evaluation of a modern structural model means solving a Bellman equation, a PDE, or a Monte Carlo simulation: minutes to hours.

And we almost never want just one evaluation:

- ▶ **Estimation** sweeps thousands of candidate parameter vectors.
- ▶ **Uncertainty quantification** propagates a distribution over parameters.
- ▶ **Policy design** optimizes over a space of policy rules.

The one advantage we have

We own the model. Unlike an econometrician with a fixed sample, we can generate as much training data as we are willing to pay for, and the labels are exact.

Look-up Tables $\rightarrow$ Neural Networks

The old idea: pre-compute once, then interpolate.

x	$\Phi(x)$
-2.0	0.0228
-1.0	0.1587
0.0	0.5000
1.0	0.8413
2.0	0.9772

The normal CDF table in the back of every old statistics textbook. Nobody re-derives Φ ; they look it up.

A 21st-century look-up table:

a network that has memorized $\mathbf{x} \mapsto y$,

$$\phi(\mathbf{x} \mid \mathbf{w}) \approx y = f(\mathbf{x}).$$



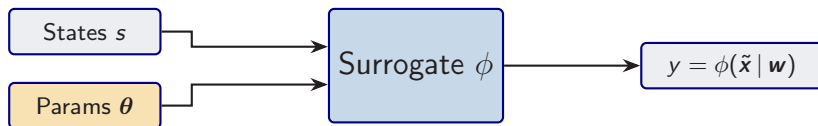
A table has one entry per grid point and dies of the curse of dimensionality. A network is **continuous**, **differentiable**, and its cost grows with the complexity of f , not with n_g^d .

Pseudo-States: The Whole Trick

Chen, Didisheim & Scheidegger (2026)

Do not build one surrogate per parameter vector. Treat the parameters as **extra state variables**:

$$\tilde{\mathbf{x}} = \underbrace{(s_1, \dots, s_n)}_{\text{states}}, \underbrace{(\theta_1, \dots, \theta_p)}_{\text{parameters}} \in \mathbb{R}^d, \quad d = n + p.$$



Solve the model **once** over the augmented space $\Rightarrow$ an instant answer for *any* parameter configuration in the box. Everything in the rest of this lecture is a different question asked of that one object.

Building One, in Four Steps

1. **Design.** Draw $\tilde{\mathbf{x}}_i$, $i = 1, \dots, m$, over the box $\prod_j [\underline{x}_j, \bar{x}_j]$: uniform, Latin hypercube, Sobol' sequence, or (Part III) actively.
2. **Label.** Evaluate the expensive model: $y_i = f(\tilde{\mathbf{x}}_i)$. **This is the step that costs money.**
3. **Train.** Minimize the empirical risk

$$\hat{\mathbf{w}} = \arg \min_{\mathbf{w}} \frac{1}{m} \sum_{i=1}^m |\phi(\tilde{\mathbf{x}}_i | \mathbf{w}) - y_i|^2.$$

4. **Validate.** Held-out max and mean absolute error, and for parametric surrogates, error as *a function of θ* , not just on average.

Where the error actually comes from

Because data is unlimited, scarcity-driven overfitting is not the binding constraint. What binds is **approximation error** (network capacity against the true regularity of f) and **label noise** when f is itself an approximate inner solver.

Not Ordinary Supervised Learning

Standard supervised learning

- ▶ Data is scarce and noisy
- ▶ Overfitting is the main enemy
- ▶ Regularization, early stopping, cross-validation
- ▶ Careful tuning, small learning rates

Surrogate modeling

- ▶ Data is **unlimited**: we own the model
- ▶ Labels are **exact** up to solver tolerance
- ▶ Large budgets, aggressive learning rates
- ▶ Focus on *approximation*, not generalization

What a trained surrogate gives you

- ▶ **Accurate enough for the downstream task**: in the option-pricing benchmark, 95th-percentile implied-vol errors below 0.13% of the mean, rising to 0.43% under a week to maturity. **Accuracy is not uniform over the box**
- ▶ **Orders of magnitude faster**: and **differentiable**: gradients in θ free, by autodiff

What the Speed-Up Buys

Table 2 of Chen, Didisheim & Scheidegger (2026), *Journal of Financial Economics*, for the Bates model:

All SPX options on an average day	FFT	Surrogate	+ GPU
Pricing, 1 day ($\approx 4,000$ options)	10 s	0.6 s	0.06 s
Gradient, 1 day	180 s	3.2 s	0.3 s
Estimation, 1 year (252 days $\times$ 10 iterations)	125 h	2.2 h	0.2 h

The point is not the speed but the new question

At 125 hours per pass, re-estimating the model every day for a year was not a slow research project; it was **not a research project at all**. At 0.2 hours it becomes an option-implied tail-risk measure with a daily time series.

Notation from Here On

This is an *estimation and policy* lecture, so one symbol changes meaning relative to this morning.

Symbol	Meaning here	This morning (DEQN deck)
θ	structural parameters (e.g. ϱ, β)	not used
ϑ	policy parameters (tax and transfer rules)	not used
$\mathbf{w}$	network weights	written θ
$\phi(\cdot \mathbf{w})$	the surrogate	$\mathcal{N}_\theta$

Why the swap

Every paper in this literature writes θ for the object being estimated; keeping θ for network weights would make the SMM criterion in Part II unreadable. **The weights are never the object of interest here**; they are an implementation detail of the look-up table.

Three Questions, One Surrogate

The rest of the lecture is the same object, post-processed three different ways.

Operator	Question	Post-processing	Where
Estimate	Which parameters fit the data?	arg min of an SMM criterion over θ	Part II
Quantify	Which parameter uncertainties matter?	propagate $p(\theta)$ through a quantity of interest: univariate effects, Sobol', Shapley	Part IV
Design	Which policy is best?	constrained arg max of a welfare objective over ϑ	Part V

The same skeleton throughout: surrogate 1 solves the equilibrium over states *and* pseudo-states; when the quantity of interest is *itself* expensive, a second surrogate maps parameters to it. **Part II needs only surrogate 1:** its moments are cheap. Parts IV and V need both, and Part III is about what surrogate 2 should be made of.

Part III

Gaussian Processes: When Data Is Expensive

When One Label Costs Four Hours

Everything so far assumed labels are cheap. For option pricing they are: 10^6 evaluations is an afternoon. **That fails the moment the model is a global stochastic equilibrium.**

What a deep network needs

- ▶ 10^4 to 10^6 labeled points
- ▶ An architecture and hyperparameters to search for
- ▶ Many epochs of SGD, and no uncertainty unless you train an ensemble

What a Gaussian process needs

- ▶ Tens to a few thousand points
- ▶ A kernel: an explicit smoothness assumption
- ▶ A closed-form posterior, no SGD, and **posterior variance everywhere, for free**

Why the variance is the real prize

It is not decoration: it tells you **where the surrogate does not know the answer**. That is what lets you spend the next expensive evaluation well, and what says whether a welfare surface is trustworthy before you optimize it.

Recall: Multivariate Gaussians

A random vector $\mathbf{x} \in \mathbb{R}^d$ is multivariate Gaussian if

$$\mathbf{x} \sim \mathcal{N}(\boldsymbol{\mu}, \Sigma),$$

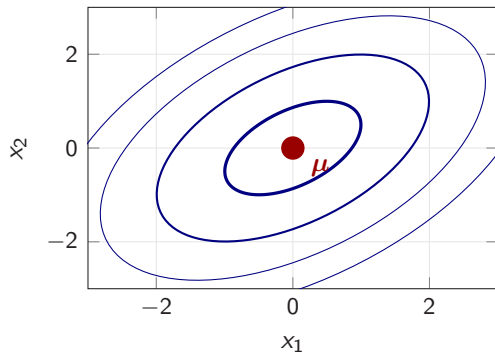
with density

$$p(\mathbf{x}) \propto \exp\left(-\frac{1}{2}(\mathbf{x} - \boldsymbol{\mu})^\top \Sigma^{-1}(\mathbf{x} - \boldsymbol{\mu})\right).$$

Key fact

It is **fully characterized** by $\boldsymbol{\mu}$ and Σ . Nothing else about the distribution is free.

2D Gaussian contours ($\rho = 0.5$)



Conditioning Is Regression

Joint:

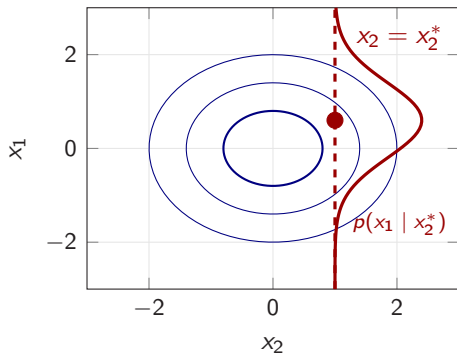
$$\begin{pmatrix} x_1 \\ x_2 \end{pmatrix} \sim \mathcal{N}\left(\begin{pmatrix} \mu_1 \\ \mu_2 \end{pmatrix}, \begin{pmatrix} \Sigma_{11} & \Sigma_{12} \\ \Sigma_{21} & \Sigma_{22} \end{pmatrix}\right)$$

Conditional:

$$\mu_{1|2} = \mu_1 + \Sigma_{12}\Sigma_{22}^{-1}(x_2 - \mu_2)$$

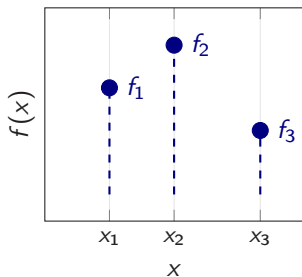
$$\Sigma_{1|2} = \Sigma_{11} - \Sigma_{12}\Sigma_{22}^{-1}\Sigma_{21}$$

These two lines are the entire engine of GP regression. Everything that follows is bookkeeping about how to build Σ .



Slicing the joint at $x_2 = x_2^*$ leaves a 1D Gaussian: a prediction *and* an error bar.

From Observations to Interpolation



Assume the function values are jointly Gaussian:

$$\begin{pmatrix} f_1 \\ f_2 \\ f_3 \end{pmatrix} \sim \mathcal{N}\left(0, \begin{pmatrix} K_{11} & K_{12} & K_{13} \\ K_{21} & K_{22} & K_{23} \\ K_{31} & K_{32} & K_{33} \end{pmatrix}\right)$$

Nearby points should be more correlated than distant ones. With a squared-exponential kernel and $\ell = 2$:

$$K = \begin{pmatrix} 1.00 & 0.76 & 0.22 \\ 0.76 & 1.00 & 0.61 \\ 0.22 & 0.61 & 1.00 \end{pmatrix}$$

Build the covariance from a **kernel**, $K_{ij} = k(x_i, x_j)$. Conditioning on the observed f_i then predicts everywhere else.

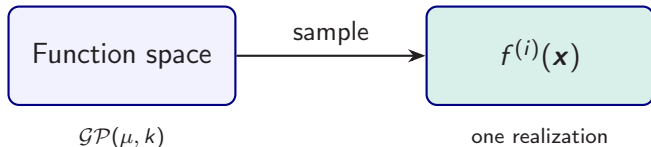
A Gaussian Process: A Distribution over Functions

Definition

A **Gaussian process** is a collection of random variables, any finite subset of which is jointly Gaussian:

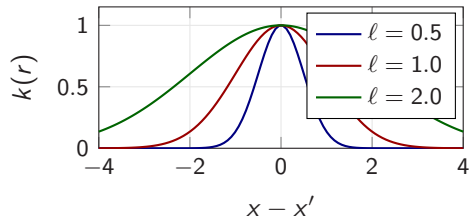
$$f(\mathbf{x}) \sim \mathcal{GP}(\mu(\mathbf{x}), k(\mathbf{x}, \mathbf{x}')).$$

- ▶ **Mean function** $\mu(\mathbf{x}) = \mathbb{E}[f(\mathbf{x})]$, almost always 0 after centering the data.
- ▶ **Covariance function** $k(\mathbf{x}, \mathbf{x}') = \mathbb{E}[(f(\mathbf{x}) - \mu(\mathbf{x}))(f(\mathbf{x}') - \mu(\mathbf{x}'))]$, where all the modeling lives.



Kernels: Where the Prior Lives

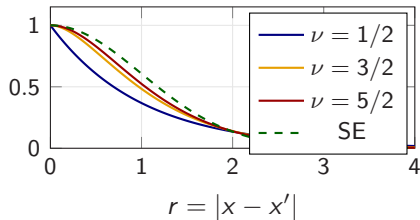
Squared exponential



$k_{\text{SE}}(r) = \sigma_f^2 \exp(-r^2/2\ell^2)$. Larger $\ell \Rightarrow$ smoother; samples are **infinitely differentiable**, often too optimistic.

Choosing the kernel is not a technicality: it *is* the model. Assume more smoothness than the truth has, and the GP interpolates confidently straight through a kink.

Matérn family, $\ell = 1$



ν sets differentiability: samples are $\lceil \nu \rceil - 1$ times differentiable. $\nu = 5/2$ is the **sane default** for value and policy functions.

GP Regression: The Posterior in Two Lines

Joint distribution of training values $\mathbf{f}$ and test values $\mathbf{f}_*$:

$$\begin{pmatrix} \mathbf{f} \\ \mathbf{f}_* \end{pmatrix} \sim \mathcal{N}\left(0, \begin{pmatrix} K & K_* \\ K_*^\top & K_{**} \end{pmatrix}\right), \quad K_{ij} = k(\mathbf{x}_i, \mathbf{x}_j).$$

Condition on what you observed: the two lines from three slides ago:

$$\mu_{*|\mathcal{D}} = K_*^\top K^{-1} \mathbf{f}$$

$$\Sigma_{*|\mathcal{D}} = K_{**} - K_*^\top K^{-1} K_*$$

Read the variance line carefully

- ▶ Near the data: $K_*^\top K^{-1} K_* \approx K_{**}$, so the variance **collapses**.
- ▶ Far from the data: $K_*^\top K^{-1} K_* \approx 0$, so the variance **reverts to the prior**.

Note what this means: the GP *knows it is extrapolating*, and says so, without ever seeing the true function out there. No trained network does this by default.

Noise, the Nugget, and Numerical Stability

Observation model. If labels carry noise, $y = f(\mathbf{x}) + \varepsilon$ with $\varepsilon \sim \mathcal{N}(0, \sigma_y^2)$, then $\text{cov}[y_p, y_q] = k(\mathbf{x}_p, \mathbf{x}_q) + \sigma_y^2 \delta_{pq}$, so everywhere replace K by $K_y = K + \sigma_y^2 I$:

$$\bar{f}_* = \mathbf{k}_*^\top (K + \sigma_y^2 I)^{-1} \mathbf{y}, \quad \sigma_*^2 = k(\mathbf{x}_*, \mathbf{x}_*) - \mathbf{k}_*^\top (K + \sigma_y^2 I)^{-1} \mathbf{k}_*.$$

Noiseless ($\sigma_y^2 = 0$)

- ▶ Interpolates the training points exactly
- ▶ Variance is zero at training inputs
- ▶ Needs K strictly positive definite

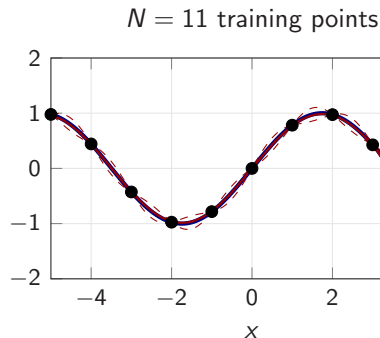
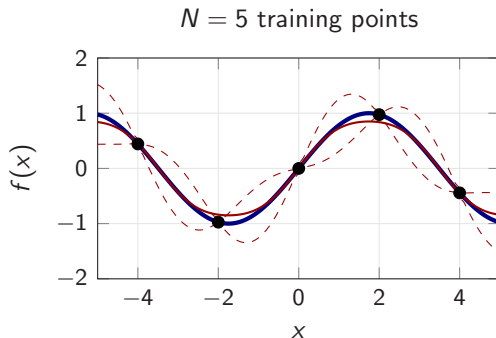
Noisy ($\sigma_y^2 > 0$)

- ▶ Smooths through the training points
- ▶ Variance positive everywhere
- ▶ Much better conditioned

In practice

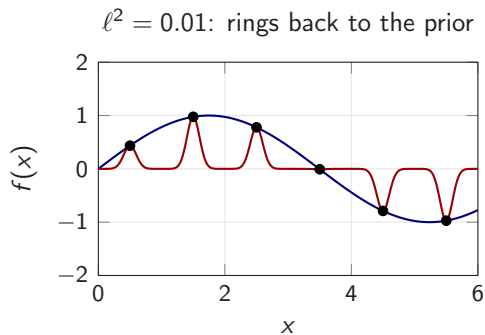
Even for a “noiseless” surrogate, add a jitter of $\sigma_y^2 \sim 10^{-6}$. **And be honest about whether it really is noiseless:** if the labels came from a Monte Carlo simulation or an iterative solver stopped at a tolerance, they carry noise, and pretending otherwise makes the Cholesky factorization fail before it makes the statistics wrong.

Prediction and Credible Bands

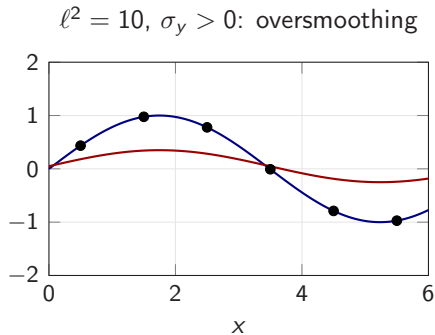


Blue: true function. **Red:** posterior mean. **Dashed:** approximate 95% credible band. The band pinches shut at each training input and balloons between them, and **that shape is the map of where to sample next**, and we use it at the end of this part.

Getting the Length Scale Wrong



Too short. The GP interpolates every point exactly and then falls back to the prior mean in between. Perfect training fit, useless predictions, and the credible band (previous slide) balloons in exactly those gaps.



Too long. The posterior mean flattens and no longer passes through the data. The band is narrow and confidently wrong, which is the more dangerous failure of the two.

Learning the Hyperparameters, and a Free Diagnostic

Marginal likelihood: how ℓ , σ_f , σ_y get chosen

$$\log p(\mathbf{y} \mid X) = \underbrace{-\frac{1}{2}\mathbf{y}^\top K_y^{-1}\mathbf{y}}_{\text{data fit}} \underbrace{-\frac{1}{2}\log |K_y|}_{\text{complexity penalty}} \underbrace{-\frac{N}{2}\log(2\pi)}_{\text{constant}}$$

The two terms pull against each other: **an automatic Occam's razor**, with closed-form gradients and no validation split spent. Optimize with restarts; it is multimodal.

Leave-one-out, for free

$$\mu_{-i}(\mathbf{x}_i) - y_i = -\frac{[K_y^{-1}\mathbf{y}]_i}{[K_y^{-1}]_{ii}}, \quad \sigma_{-i}^2(\mathbf{x}_i) = \frac{1}{[K_y^{-1}]_{ii}}.$$

These need $\text{diag}(K_y^{-1})$, so *all* N leave-one-out residuals cost one further $\mathcal{O}(N^3)$ solve, the same order you already paid, and **no refitting and no held-out data**. Doing N honest refits would be $\mathcal{O}(N^4)$. **This is the number to report for a surrogate**; Part V quotes it.

Closed form: Rasmussen & Williams (2006), §5.4.2, in ../readings.

GP Regression in Fifteen Lines

```
import numpy as np
from scipy.linalg import cho_factor, cho_solve

def kernel(X1, X2, l=1.0, sf=1.0):
    sq = np.sum(X1**2, 1).reshape(-1, 1) \
        + np.sum(X2**2, 1) - 2 * X1 @ X2.T
    return sf**2 * np.exp(-0.5 * sq / l**2)

K = kernel(X_tr, X_tr) + 1e-8 * np.eye(N)      # jitter
L, low = cho_factor(K, lower=True)             #  $O(N^3)$ , once
alpha = cho_solve((L, low), y_tr)              # the weights

Ks = kernel(X_tr, X_te)
mu = Ks.T @ alpha                             # posterior mean
v = cho_solve((L, low), Ks)
var = kernel(X_te, X_te) - Ks.T @ v           # posterior variance
```

No training loop, no learning rate, no architecture: the whole model is the kernel and one Cholesky factorization. Compare Algorithm 2.1 in Rasmussen & Williams (2006); notebook 06_02 builds this and checks it against sklearn.

What It Costs: $\mathcal{O}(N^3)$

Cost of exact GP inference

- ▶ **Fitting:** Cholesky of $K_y \in \mathbb{R}^{N \times N}$: $\mathcal{O}(N^3)$ time, $\mathcal{O}(N^2)$ memory.
- ▶ **Prediction:** $\mathcal{O}(N)$ per test point (mean), $\mathcal{O}(N^2)$ (variance).
- ▶ **Hyperparameters:** $\mathcal{O}(N^3)$ per step.

If you really need large N

Sparse GPs (inducing points, rank $M \ll N$, cost $\mathcal{O}(NM^2)$), random Fourier features, GPyTorch.

But notice where this bites

The cubic cost sounds fatal until you ask *why* N is small. If a label costs a four-hour solve, you were never reaching $N = 10^4$ anyway. The regime where GPs are cheap to fit is the regime where data is expensive to get.

Gaussian Processes versus Deep Networks

Criterion	Gaussian process	Deep neural network
Uncertainty	✓built in (posterior variance)	✗needs ensembles or MC dropout
Hyperparameters	✓marginal likelihood	●search: random, Hyperband
Scaling in N	✗ $\mathcal{O}(N^3)$, practical limit $\sim 10^4$	✓ $\mathcal{O}(N)$ per epoch
Scaling in d	✗degrades past $d \approx 10$	✓handles $d \gg 100$
Small data ($N < 100$)	✓excellent	✗underfits, or memorizes
Fitting	✓closed form	●SGD, many epochs, seed-dependent
Interpretability	✓the kernel <i>is</i> the assumption	✗black box

The decision rule is the *cost of a label*, not taste

- ▶ **Labels cheap, d large:** deep network, as in Part II's option-pricing surrogate.
- ▶ **Labels expensive, $d \lesssim 10$:** GP plus active learning, as in Parts IV and V. (The table's "degrades past $d \approx 10$ " is the same boundary.)
- ▶ **Both, layered:** a DEQN solves the equilibrium, a GP maps its output to the QoI.

Finger Exercise: How Many Points?

PAUSE: 5 min

The setting

You must map a welfare objective over a **6-dimensional** policy box. One evaluation means simulating a stochastic OLG model, ≈ 4 hours of wall-clock time. You have a 64-core machine and **two weeks**.

1. Roughly how many evaluations can you afford?
2. How many points per dimension would a tensor grid give you at that budget?
3. Which approximator does that budget select, and why?

Solution: How Many Points?

1. The budget

14 days $\times$ 24 h = 336 h of wall-clock time; at 64 cores that is $\sim 21,500$ core-hours, so about **5,000 evaluations** at 4 h each. Call it a few thousand, allowing for failures and re-runs.

2. What a grid does with them

$5,376^{1/6} \approx 4.2$ points per dimension. Four per axis will not resolve a curved welfare surface, and refining to a still-inadequate six costs $6^6 = 46,656$ evaluations = 187,000 core-hours, or **four months** on the same 64 cores (21 years on one).

3. Which approximator

A few thousand points is far too few to train a deep network, and comfortably inside the $\mathcal{O}(N^3)$ range of an exact GP. **Choose the GP**, and since every point is precious, do not place them on a grid at all. Let the posterior variance choose them.

The real study in Part V spent **500**, not 5,000. The next three slides are how.

Where to Sample Next

Fixed design

- ▶ Spread evenly, fixed before you know anything
- ▶ Wasted where the function is flat

Active design

- ▶ Goes where the surrogate is **uncertain**
- ▶ Reuses the posterior variance you already have

The score function

$$U(\mathbf{x}) = w_{\text{obj}} \mu(\mathbf{x}) + \frac{w_{\text{var}}}{2} \log \sigma^2(\mathbf{x})$$

- ▶ $\sigma^2(\mathbf{x})$, **exploration**: sample where the variance is large.
- ▶ $\mu(\mathbf{x})$, **objective-following**: for Bayesian *optimization*.

In practice Part V uses the upper-confidence-bound form $\alpha = \lambda\mu + \kappa\sigma$, $\lambda = 1$, $\kappa = 100$: exploration-weighted, but not pure, because there the optimum *is* the target. **And note what σ^2 does not depend on**: with hyperparameters fixed it is a function of the *design* alone, for any kernel. Refit ℓ each step and the data creep back in; two slides on, that matters.

The BAL Algorithm

Bayesian active learning for surrogates

Require: initial design $\mathcal{D}_0 = \{(\mathbf{x}_i, y_i)\}_{i=1}^{n_0}$, budget N , candidate set $\mathcal{C}$

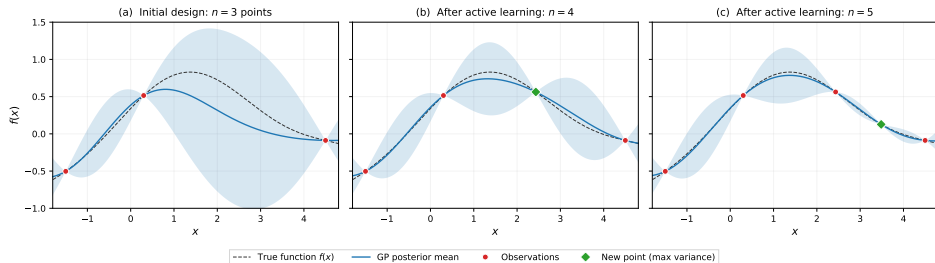
- 1: fit a GP on $\mathcal{D}_0$, giving $\mu(\cdot)$ and $\sigma^2(\cdot)$
- 2: **for** $t = 1, \dots, N - n_0$ **do**
- 3: evaluate $U(\mathbf{x}) = w_{\text{obj}} \mu(\mathbf{x}) + \frac{w_{\text{var}}}{2} \log \sigma^2(\mathbf{x})$ for all $\mathbf{x} \in \mathcal{C}$
- 4: select $\mathbf{x}_{\text{new}} = \arg \max_{\mathbf{x} \in \mathcal{C}} U(\mathbf{x})$
- 5: **query the expensive model:** $y_{\text{new}} = f(\mathbf{x}_{\text{new}})$
- 6: update $\mathcal{D}_t \leftarrow \mathcal{D}_{t-1} \cup \{(\mathbf{x}_{\text{new}}, y_{\text{new}})\}$ and refit the GP
- 7: **end for**
- 8: **return** the trained GP surrogate

Scheidegger & Bilonis (2019), in ../readings, puts this loop to work on economic models.

Every step except line 5 is cheap; the loop exists to make line 5 happen as rarely as possible, and in a real study to tolerate its failure.

BAL in Action

Each new point goes where the posterior variance is largest (the marked point).



(a) three observations, the posterior drifting where there is no data. (b) the acquisition takes the max-variance point. (c) one more fills the gap: **five points suffice**.

But do not oversell it. Notebook 06_02 runs this against a uniform design and they finish *level* (MAE 0.0234 vs 0.0237 at $N = 25$). For a *stationary* kernel the posterior variance depends on the design alone, so on an interval uncertainty sampling essentially *is* a space-filling design. It earns its keep where no such design exists: **irregular domains, ARD kernels, and dimensions where a grid is impossible**.

Part VI

Synthesis and Hands-On

One Surrogate, Three Questions

	Part II: estimate	Part IV: quantify	Part V: design
Model	Brock–Mirman	stochastic IAM with Bayesian learning about the climate sensitivity	stochastic OLG IAM with tipping risk
Surrogate 1 inputs	(z, K, β, ϱ)	climate-economy states plus uncertain θ	OLG state plus 16 policy pseudo-states
Qol	moments m_1 to m_3	SCC(θ)	40 cohort welfares
Surrogate 2	none needed	GP + active learning	40 GPs + active learning
Operator	arg min under common random numbers	global sensitivity: univariate effects, Sobol', Shapley	constrained arg max
Output Headline	an interior estimate a flat direction is the moments <i>or</i> the surrogate	an importance ranking climate sensitivity and damages drive the SCC	a Pareto-improving tax 0.42% gain, and the tail is cut

The Toolbox: When to Use What

Method	Use when	Watch out for
Polynomials, splines	$d \leq 3$, smooth, cheap labels	no kinks, no irregular domains
Sparse grids	$d \lesssim 20$, local features matter	tensor structure, hypercube domain
Gaussian process	labels expensive, $d \lesssim 10$	$\mathcal{O}(N^3)$; the kernel <i>is</i> the model
Deep network	labels cheap, d large	no uncertainty; architecture search
GP + active learning	every evaluation is precious	sequential, so hard to parallelize

A decision procedure, in order

1. **Price one label.** Seconds, or hours? That number decides more than anything else here.
2. **Count the dimensions** *after* promoting parameters to pseudo-states, not before.
3. **Ask whether you need an error bar.** If a decision rides on the surrogate, you do.
4. **Layer them** when model and quantity of interest have different cost profiles, the common case.

Practical Guidance

- ▶ **Train once, then simulate for free.** Surrogate 1 costs hours from scratch and minutes warm-started. Every downstream question is then a forward pass. Budget accordingly: the expensive part is the design, not the training.
- ▶ **Validate *in the parameter direction*, not on average.** A surrogate with excellent mean error can still be nearly flat in θ : low residual, wrong policy. This is **lazy learning**; the cure (anchors and distillation) is Part VI of this morning's deck. **Do not skip this check**; it is the most common way a surrogate study goes quietly wrong.
- ▶ **Report leave-one-out error, not training error:** and report the **maximum** over the parameter box, not the mean. The maximum is what bounds the bias in θ^* .
- ▶ **Use common random numbers whenever you sweep a parameter.** Without them you are optimizing simulation noise, in estimation *and* in policy design.
- ▶ **When one quantity-of-interest evaluation is expensive, add the second surrogate.** Expected welfare at one policy took 10,000 Monte Carlo paths; the GP layer cut the whole study to 500 simulations at leave-one-out error 3.2×10^{-5} .

Hands-On: What to Run

Notebooks in `../code/06_surrogates_and_gps` and `../code/07_structural_estimation`: PyTorch + scikit-learn, CPU-only, self-contained. 06_01 and 06_02 ship on `RUN_MODE = "smoke"`; raise it once a run works.

- ▶ **A deep surrogate, end to end** (≈ 3.5 min). Black–Scholes over (S, K, T, σ, r) , then implied-vol inversion by root-finding *and* by gradient descent. **Read its timing table:** on this toy the surrogate *loses* to numpy: the “price one label first” rule biting. 06_01
- ▶ **GP regression and active learning** (≈ 30 s). A GP from scratch in numpy, checked against sklearn; then the BAL loop and its budget sweep. 06_02
- ▶ **Estimating ϱ by SMM** (≈ 10 s). Train with ϱ as a pseudo-state, simulate under common random numbers, recover the parameter. 07_01
- ▶ **The identification ridge** (≈ 10 s). Free β as well, watch the valley appear, and read the moment Jacobian’s singular values. 07_02

If you only run one: 06_02. It builds a GP from scratch, and its budget sweep is the measurement behind the honest version of the active-learning claim.



Questions?

Simon Scheidegger

HEC, University of Lausanne

simon.scheidegger@unil.ch · sischei.github.io

Next up, Session 7: *Structural estimation via the simulated method of moments*, where the surrogate we just built removes the inner-loop model solve.

Materials: github.com/sischei/deep_learning_padova_0926

Cloud: nuvolos.cloud